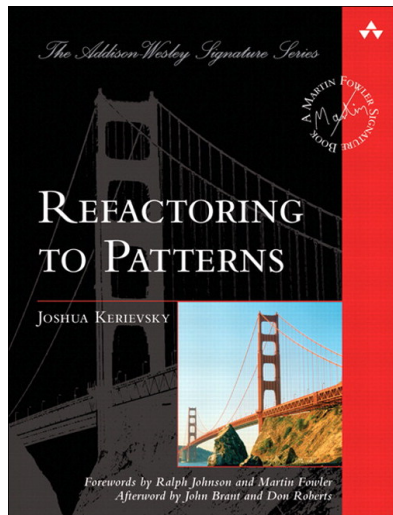


High Level Refactoring Patterns (SB5-MAI)

Jan Corfixen Sørensen

University of Southern Denmark

Introduction



- ▶ Refactoring to Patterns
- ▶ High Level Catalog
- ▶ Joshua Kerievsky

Patterns Happy?

```
interface MessageStrategy {  
    public void sendMessage();  
}  
  
abstract class AbstractStrategyFactory {  
    public abstract MessageStrategy createStrategy(MessageBody mb);  
}  
  
class MessageBody {  
    Object payload;  
    public Object getPayload() {  
        return payload;  
    }  
    public void configure(Object obj) {  
        payload = obj;  
    }  
    public void send(MessageStrategy ms) {  
        ms.sendMessage();  
    }  
}
```

Patterns Happy?

```
class DefaultFactory extends AbstractStrategyFactory {
    private DefaultFactory() {
    }
    static DefaultFactory instance;
    public static AbstractStrategyFactory getInstance() {
        if (instance == null)
            instance = new DefaultFactory();
        return instance;
    }

    public MessageStrategy createStrategy(final MessageBody mb) {
        return new MessageStrategy() {
            MessageBody body = mb;
            public void sendMessage() {
                Object obj = body.getPayload();
                System.out.println(obj);
            }
        };
    }
}
```

Patterns Happy?

```
public class HelloWorld {  
    public static void main(String[] args) {  
        MessageBody mb = new MessageBody();  
        mb.configure("Hello World!");  
        AbstractStrategyFactory asf = DefaultFactory.getInstance();  
        MessageStrategy strategy = asf.createStrategy(mb);  
        mb.send(strategy);  
    }  
}
```

Revision of The Declaration of Independence

In 1776, Thomas Jefferson wrote the following un-famous words:

“We hold these truths to be sacred and undeniable...”

In 1776, Benjamin Franklin revised Jefferson's words to read:

“We hold these truths to be self-evident...”

What Is A Pattern?

Each pattern is a three-part rule, which expresses a relation between a certain context, a problem, and a solution.

As an element in the world, each pattern is a relationship between a certain context, a certain system of forces which occurs repeatedly in that context, and a certain spatial configuration which allows these forces to resolve themselves.

As an element of language, a pattern is an instruction, which shows how this spatial configuration can be used, over and over again, to resolve the given system of forces, wherever the context makes it relevant.

The pattern is, in short, at the same time a thing, which happens in the world, and the rule which tells us how to create that thing, and when we must create it. It is both a process and a thing; both a description of a thing which is alive, and a description of the process which will generate that thing.

[Alexander, ATWoB, p247]

Algebra and Word Problems

In algebra class, we first learn different manipulations, like:

“add the same value to both sides of the equation”

“the commutative property of addition allows us to swap its operands.”

Once we know the manipulations, we're given word problems:

“A train leaves New York heading West. . . .”

To solve this problem, you express it in terms of an algebraic equation and then apply the rules of algebra to arrive at an answer.

Refactoring and Patterns

Design patterns are the word problems of the programming world; refactoring is its algebra. After having read Design Patterns [DP], you reach a point where you say to yourself, “If I had only known this pattern, my system would be so much cleaner today.” The book you are holding introduces you to several sample problems, with solutions expressed in the operations of refactoring.

The Algebra of Refactoring

Many people will read this book and try to memorize the steps to implement these patterns. Others will read this book and clamor for these larger refactorings to be added to existing programming tools. Both of these approaches are misguided. **The true value of this book lies not in the actual steps to achieve a particular pattern but in understanding the thought processes that lead to those steps.** By learning to think in the algebra of refactoring, you learn to solve design problems in behavior-preserving steps, and you are not bound by the small subset of actual problems that this book represents.

Patterns of Refactoring

So take these exemplars that Josh has laid out for you. Study them. **Find the underlying patterns of refactoring that are occurring.** Seek the insights that led to the particular steps. Don't use this as a reference book, but as a primer.

Code Smell to Refactoring (1)

Smell	Refactoring
Alternative Classes with Different Interfaces (43) [F]	Unify Interfaces with Adapter (247)
Combinatorial Explosion (45)	Replace Implicit Language with Interpreter (269)
Conditional Complexity (41)	Replace Conditional Logic with Strategy (129) Move Embellishment to Decorator (144) Replace State-Altering Conditionals with State (166) Introduce Null Object (301)
Duplicated Code (39) [F]	Form Template Method (205) Introduce Polymorphic Creation with Factory Method (88) Chain Constructors (340) Replace One/Many Distinctions with Composite (224) Extract Composite (214) Unify Interfaces with Adapter (247) Introduce Null Object (301)
Indecent Exposure (42)	Encapsulate Classes with Factory (80)
Large Class (44) [F]	Replace Conditional Dispatcher with Command (191) Replace State-Altering Conditionals with State (166) Replace Implicit Language with Interpreter (269)
Lazy Class (43) [F]	Inline Singleton (114)

Code Smell to Refactoring (2)

Long Method (40) [F]	Compose Method (123)
	Move Accumulation to Collecting Parameter (313)
	Replace Conditional Dispatcher with Command (191)
	Move Accumulation to Visitor (320)
	Replace Conditional Logic with Strategy (129)
Oddball Solution (45)	Unify Interfaces with Adapter (247)
Primitive Obsession (41) [F]	Replace Type Code with Class (286)
	Replace State-Altering Conditionals with State (166)
	Replace Conditional Logic with Strategy (129)
	Replace Implicit Tree with Composite (178)
	Replace Implicit Language with Interpreter (269)
	Move Embellishment to Decorator (144)
Solution Sprawl (43)	Encapsulate Composite with Builder (96)
	Move Creation Knowledge to Factory (68)
Switch Statements (44) [F]	Replace Conditional Dispatcher with Command (191)
	Move Accumulation to Visitor (320)

A Study Sequence (1)

Session	Refactoring(s)
1	Replace Constructors with Creation Methods (57) Chain Constructors (340)
2	Encapsulate Classes with Factory (80)
3	Introduce Polymorphic Creation with Factory Method (88)
4	Replace Conditional Logic with Strategy (129)
5	Form Template Method (205)
6	Compose Method (123)
7	Replace Implicit Tree with Composite (178)
8	Encapsulate Composite with Builder (96)
9	Move Accumulation to Collecting Parameter (313)
10	Extract Composite (214) Replace One/Many Distinctions with Composite (224)
11	Replace Conditional Dispatcher with Command (191)

A Study Sequence (2)

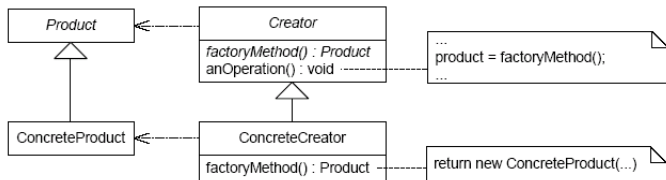
11	Replace Conditional Dispatcher with Command (191)
12	Extract Adapter (258) Unify Interfaces with Adapter (247)
13	Replace Type Code with Class (286)
14	Replace State-Altering Conditionals with State (166)
15	Introduce Null Object (301)
16	Inline Singleton (114) Limit Instantiation with Singleton (296)
17	Replace Hard-Coded Notifications with Observer (236)
18	Move Embellishment to Decorator (144) Unify Interfaces (343) Extract Parameter (346)
19	Move Creation Knowledge to Factory (68)
20	Move Accumulation to Visitor (320)
21	Replace Implicit Language with Interpreter (269)

Benefits of Composite Refactorings

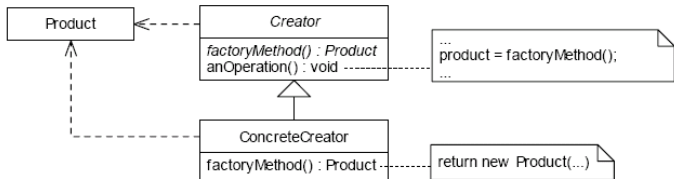
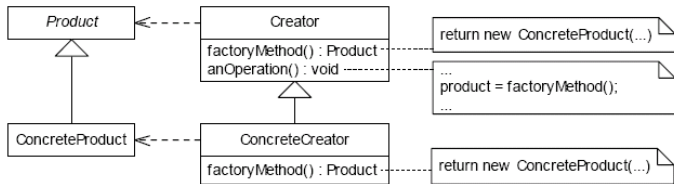
- They provide an overall plan for a refactoring sequence.
- They suggest non-obvious design directions.
- They provide insights into implementing patterns.

Pattern: Factory Method

Structure:



There Are Many Ways To Implement A Pattern!



There Are Many Ways To Implement A Pattern

It seems you can't overemphasize that a pattern's Structure diagram is just an example, not a specification. It portrays the implementation we see most often. As such the Structure diagram will probably have a lot in common with your own implementation, but differences are inevitable and actually desirable. At the very least you will rename the participants as appropriate for your domain. Vary the implementation trade-offs, and your implementation might start looking a lot different from the Structure diagram. [Vlissides, C++ Report, April 1998]

Patterns of Refactoring

Automation First

Manual refactorings are dirt roads. Automated refactorings are highways. When deciding how to refactor, look first for the highways.

Client First

We like to refactor smelly code – yet we may only see a manual way to refactor. To find a simpler, automated way of refactoring, consider starting with a client of the smelly code.

Refactoring Directions (1)

Pattern	To	Towards	Away
Adapter	Extract Adapter (258), Unify Interfaces with Adapter (247)	Unify Interfaces with Adapter (247)	
Builder	Encapsulate Composite with Builder (96)		
Collecting Parameter	Move Accumulation to Collecting Parameter (313)		
Command	Replace Conditional Dispatcher with Command (191)	Replace Conditional Dispatcher with Command (191)	
Composed Method	Compose Method (123)		
Composite	Replace One/Many Distinctions with Composite (224), Extract Composite (214), Replace Implicit Tree with Composite (178)		Encapsulate Composite with Builder (96)
Creation Method	Replace Constructors with Creation Methods (57)		
Decorator	Move Embellishment to Decorator (144)	Move Embellishment to Decorator (144)	
Factory	Move Creation Knowledge to Factory (68), Encapsulate Classes with Factory (80)		
Factory Method	Introduce Polymorphic Creation with Factory Method (88)		

Refactoring Directions (2)

Interpreter	Replace Implicit Language with Interpreter (269)	
Iterator		Move Accumulation to Visitor (320)
Null Object	Introduce Null Object (301)	
Observer	Replace Hard-Coded Notifications with Observer (236)	Replace Hard-Coded Notifications with Observer (236)
Singleton	Limit Instantiation with Singleton (296)	Inline Singleton (114)
State	Replace State-Altering Conditionals with State (166)	Replace State-Altering Conditionals with State (166)
Strategy	Replace Conditional Logic with Strategy (129)	Replace Conditional Logic with Strategy (129)
Template Method	Form Template Method (205)	
Visitor	Move Accumulation to Visitor (320)	Move Accumulation to Visitor (320)